

Cold-Start SFT

RL 직전 단계를 제대로 이해하기

이 순서로 살펴봅니다

★ RL 식에서 SFT를 유도하고, 콜드 스타트가 RL을 어떻게 정하는지 봅니다

1

Background

SFT와 RL은 같은 식의 두 극단

policy gradient에서 SFT loss를 유도하고, forward KL 관점으로 읽습니다.

2

Why Cold Start

신호가 0인 RL

GRPO 그룹 안에서 성공과 실패가 섞여야 학습 신호가 생기는 이유를 계산합니다.

3

Kimi K3

콜드 스타트 정책의 정의

K3 보고서가 SFT를 'RL의 출발 정책'으로 정의한 방식과 길이 예산 규칙을 봅니다.

4

Data & Code

Nemotron 기준 구현

궤적 포맷, loss mask, packing, mask rolling을 코드까지 따라갑니다.

5

Deep Dive

콜드 스타트가 RL을 정하는 방식

pass@k 천장, entropy, 교사-학생 격차, 체크포인트 선택을 다룹니다.

최신 레시피는 모두 콜드 스타트로 시작합니다

◆ Lambert 강의에서 SFT는 '질문-답변 형식을 익힌다' 한 줄이었습니다



4.1.1 Supervised Fine-Tuning

The SFT stage establishes a high-quality cold-start policy for the subsequent RL stage. Building on the SFT pipeline of previous Kimi models [59, 60], we expand the SFT dataset for Kimi K3, substantially broadening its coverage of complex agentic tasks. Specifically, we synthesize data trajectories using domain-specialized models from the prior Kimi series, followed by multi-stage verification and human-in-the-loop annotation. To represent these complex agentic trajectories consistently, we serialize all data with our X_TML-based chat template (eXtensible Token Markup Language; see § F for details). Collectively, these steps yield a large-scale instruction dataset that endows Kimi K3 with adaptive reasoning, precise tool calling, and robust execution in long-horizon agentic scenarios. In addition, we apply quantization-aware training (QAT) from the SFT stage onward, with MXFP4 weights and MXFP8 activations (§ 4.1.4).

핵심은 세 가지입니다

❖ 콜드 스타트 SFT는 잘하는 모델이 아니라 RL이 가르칠 수 있는 모델을 만듭니다

1

Initial policy

콜드 스타트 SFT는 RL을 시작할 **초기 정책**을 만드는 SFT입니다.

성능보다 출발점의 성질이 중요합니다.

2

Learnable signal

RL은 그룹 안의 **성공과 실패 차이**에서 배웁니다.

성공률이 0도 1도 아니어야 하고, 형식을 지키고, 다양성이 살아 있어야 합니다.

3

Same equation

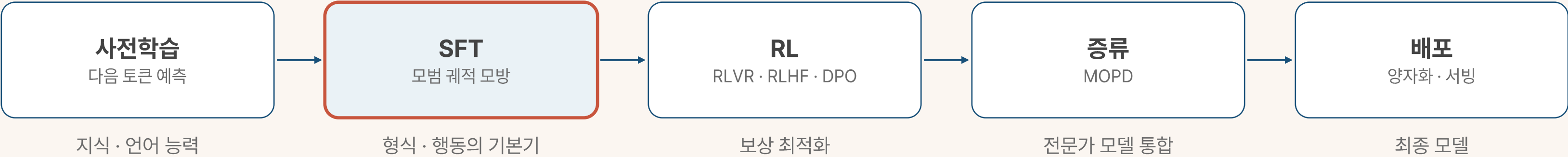
SFT gradient = **보상을 1로 고정한 off-policy policy gradient.**

두 방법은 같은 식의 두 극단입니다.

Keywords : initial policy · learnable signal · loss masking

SFT는 사전학습과 RL 사이의 모방 단계

기법은 같은 다음 토큰 예측, 달라지는 것은 데이터와 loss를 거는 위치입니다



SFT가 사전학습과 다른 점

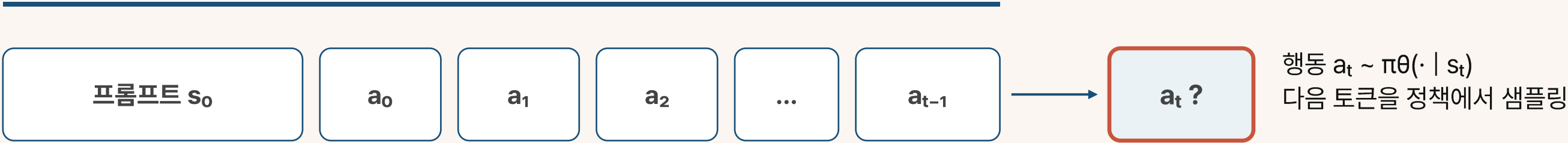
	사전학습	SFT
데이터	인터넷 규모 텍스트	원하는 행동의 모범 대화 · 궤적
loss 위치	모든 토큰	모델이 말해야 하는 토큰만 (loss mask)
배우는 것	세상 지식, 언어	형식, 역할, 도구 사용 방식

RL 모델로 좋은 답만 골라 다시 SFT하는 루프(거절 샘플링)도 흔합니다. 파이프라인은 한 방향이 아닙니다.

언어모델을 정책으로 읽기

★ 강의 노트의 policy gradient 표기를 그대로 씁니다

상태 s_t = 지금까지의 prefix ($s_0, a_0, \dots, a_{t-1}$)



기호	RL 용어	언어모델에서의 뜻
a_t	action	시점 t에 고르는 다음 토큰
s_t	state	프롬프트 + 지금까지 생성한 토큰
$\pi_{\theta}(\cdot s_t)$	policy	모델이 주는 다음 토큰 분포 (θ = 가중치)
τ	trajectory	응답 하나 전체 (rollout, episode)
$R(\tau)$	reward	응답 전체에 한 번 매기는 점수

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)]$$

모델이 뽑은 응답들의
평균 점수를 최대화

정책경사: 점수만큼 로그확률을 밀어 올린다

★ REINFORCE는 궤적의 모든 토큰을 보상 $R(\tau)$ 로 가중합니다

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R(\tau) \right]$$

식을 말로 읽기

- 모델이 **직접** 응답 τ 를 뽑습니다 ($\tau \sim \pi_{\theta}$).
- 응답의 각 토큰마다 '이 토큰의 로그확률을 올리는 방향'을 구합니다.
- 그 방향에 응답 점수 $R(\tau)$ 를 곱해 더합니다.
- $R > 0$ 이면 올리고, $R < 0$ 이면 내립니다. $|R|$ 이 클수록 큰 걸음입니다.

유도 (log-derivative trick)

$$\begin{aligned} \nabla J &= \nabla \sum_{\tau} P(\tau | \theta) R(\tau) \\ &= \sum_{\tau} P(\tau | \theta) \nabla \log P(\tau | \theta) R(\tau) \\ &= \mathbb{E}_{\tau} [\nabla \log P(\tau | \theta) R(\tau)] \\ &= \mathbb{E}_{\tau} [\sum_t \nabla \log \pi_{\theta}(a_t | s_t) R(\tau)] \end{aligned}$$

핵심: $\nabla P = P \nabla \log P$, 그리고 궤적 확률의 로그는 토큰별 로그확률의 합 (환경 전이는 θ 와 무관)

강의 노트: "this is basically the same update as SFT, but data τ is sampled from the policy and the gradient is weighted by $R(\tau)$ "

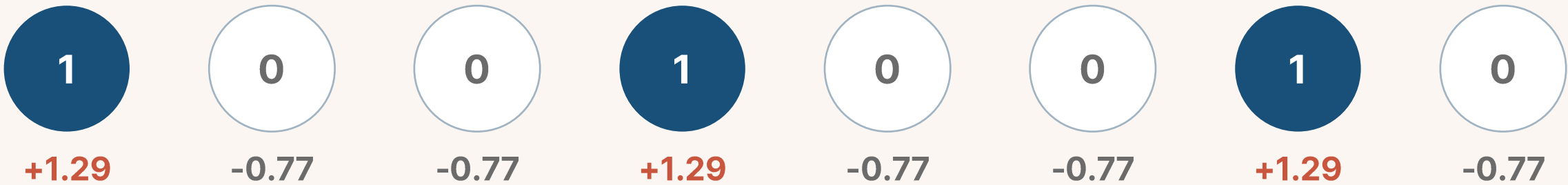
GRPO는 그룹 평균을 기준으로 씁니다

★ 같은 문제에서 G개를 뽑아, 평균보다 잘한 답만 올립니다

$$A^{(i)} = \frac{r^{(i)} - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r}) + \epsilon}$$

baseline $b(s_t)$ 가 행동에 의존하지 않으면 $E[\nabla \log \pi \cdot b] = 0 \rightarrow$ 편향 없이 분산만 줄임
GRPO는 learned value 대신 **G개 샘플의 평균**을 직접 측정합니다

한 문제에서 G = 8개를 뽑은 예



평균 = $3/8 = 0.375$
표준편차 = 0.484
위 숫자 = 보상, 아래 = A

	성공한 답 (보상 1)	실패한 답 (보상 0)
어드밴티지	$(1 - 0.375) / 0.484 = +1.29$	$(0 - 0.375) / 0.484 = -0.77$
업데이트	모든 토큰의 확률을 올림	모든 토큰의 확률을 내림

Dr. GRPO: std로 나누면 너무 쉽거나 어려운 문제의 신호가 증폭되고, 길이로 나누면 긴 오답이 덜 벌받는 편향이 생깁니다.

SFT는 보상을 1로 고정한 정책경사

★ 두 gradient는 '데이터 출처'와 '토큰 가중치' 두 곳만 다릅니다

RL (policy gradient)

$$\nabla_{\theta} J = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A_t \right]$$

① 궤적 출처: $\tau \sim \pi_{\theta}$ vs $\tau \sim D$

② 토큰 가중치: A_t vs $m_t \cdot 1$

m_t = 이 토큰에 loss를 걸면 1, 아니면 0

SFT

$$-\nabla_{\theta} \mathcal{L}_{\text{SFT}} \propto \mathbb{E}_{\tau \sim D} \left[\sum_t m_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot 1 \right]$$

	RL (policy gradient)	SFT
① 궤적은 누가	현재 모델이 생성 (on-policy)	교사 · 사람이 생성, 고정 (off-policy)
② 토큰 가중치	어드밴티지 A_t (음수면 내림)	항상 +1 (마스크된 토큰은 전부 올림)
나쁜 행동	음수 가중치로 직접 억제	데이터에 없을 뿐, 억제하지 못함

궤적을 모범 데이터로 바꾸고 어드밴티지를 모두 1로 두면 policy gradient가 SFT가 됩니다.
그래서 SFT는 '남이 만든, 전부 만점짜리 궤적으로 하는 RL'로 볼 수 있습니다.

SFT loss는 마스크된 토큰의 평균 NLL

★ m_t 가 '모델이 책임지는 토큰'을 정의합니다

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{N_m} \sum_{\tau \in B} \sum_t m_t \log \pi_{\theta}(a_t | s_t), \quad N_m = \sum_{\tau, t} m_t$$

기호를 말로 읽기

- B: 배치 안의 궤적들, τ : 궤적 하나
- m_t : 이 토큰에서 loss를 계산하면 1, 무시하면 0
- $\log \pi_{\theta}(a_t | s_t)$: 앞 내용이 주어졌을 때 정답 토큰에 모델이 준 확률의 로그
- N_m : loss를 계산한 토큰 수 → **토큰 단위 평균**

에이전트 궤적에서 m_t

역할	m_t	이유
system	0	주어지는 지시
user	0	주어지는 질문
assistant 추론	1	모델이 생성
tool_call	1	모델이 생성
tool 결과	0	환경이 생성
턴 종료 토큰	1	멈추는 법

tool 결과에 loss를 걸면 모델이 **환경의 출력을 지어내는 법**을 배웁니다.
턴 종료 토큰을 빼면 모델이 멈추지 못해 RL rollout이 최대 길이까지 늘어납니다.

같은 방향, 다른 데이터와 가중치

✦ SFT와 RL의 차이를 한 표로 정리합니다

	SFT	RL (policy gradient)
궤적 생성	교사 · 사람 (고정 데이터셋)	현재 모델 (매 step 새로 생성)
토큰 가중치	+1 (마스크된 토큰)	어드밴티지 A_t
신호의 밀도	모든 토큰에 정답이 있음 (dense)	응답 전체에 점수 하나 (sparse)
나쁜 행동	억제 불가 (데이터에 없을 뿐)	음수 어드밴티지로 억제
학습 분포	정답 prefix만 봄 → exposure bias	자기 분포에서 학습
배우는 것	모방 (데이터 수준이 천장)	개선 (보상이 허락하는 한)
비용	싸고 안정적	rollout 생성으로 비싸고 불안정

콜드 스타트 = SFT로 **dense**하고 **싼 신호**를 먼저 쓰고, RL로 **자기 분포에서의 개선**을 얻는 순서 전략입니다.

SFT는 데이터 전체를 덮으려 합니다

★ NLL 최소화 = forward KL 최소화 → mass-covering

$$\mathbb{E}_{y \sim p_{\text{data}}}[-\log \pi_{\theta}(y)] = D_{\text{KL}}(p_{\text{data}} \parallel \pi_{\theta}) + H(p_{\text{data}}) \quad \leftarrow \text{SFT}$$

$$\mathcal{J}_{\text{RLHF}} = \mathbb{E}_{y \sim \pi_{\theta}}[R(y)] - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) \quad \leftarrow \text{RL} \cdot \text{RLHF}$$

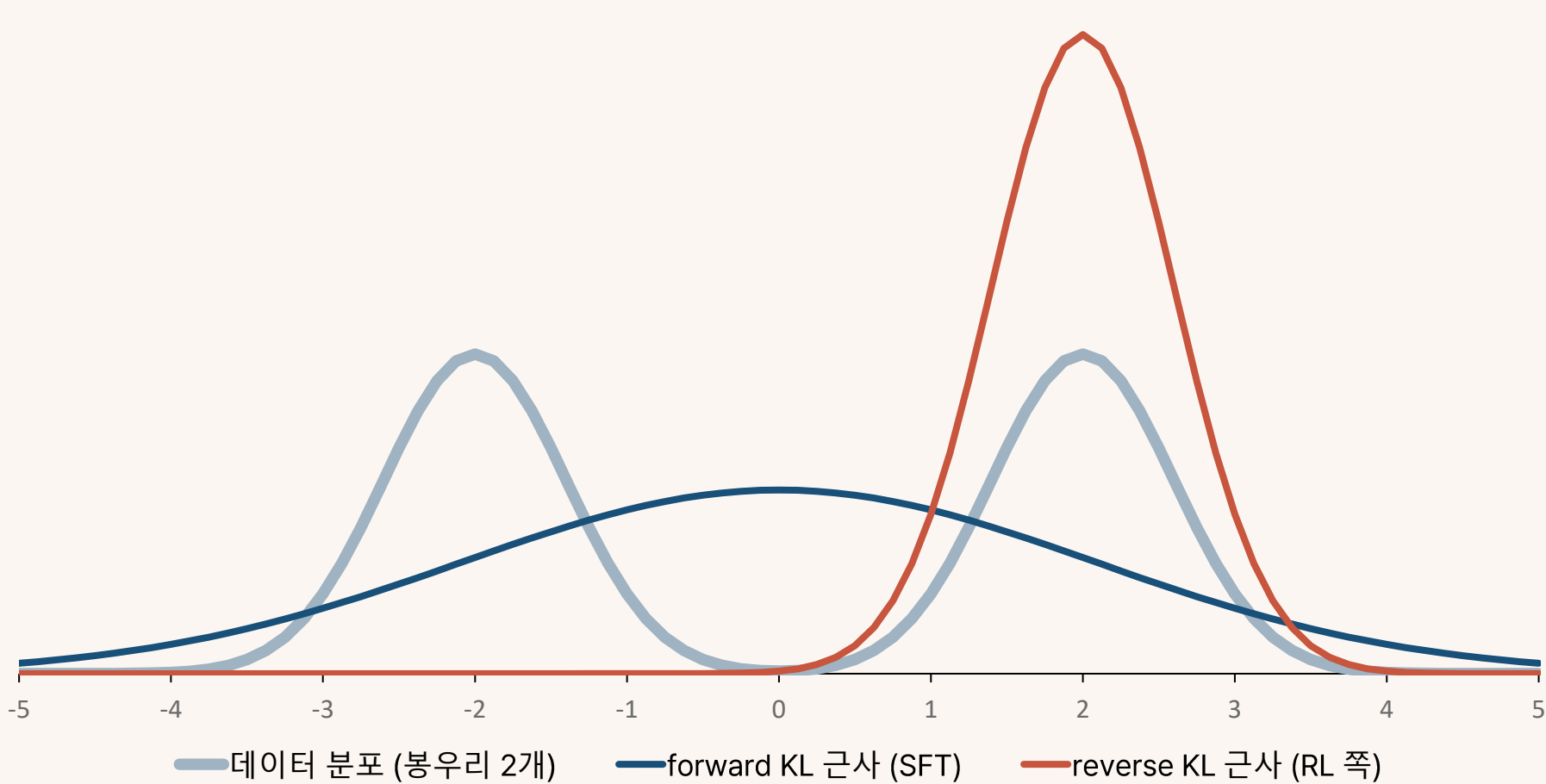


차트: 봉우리 2개인 분포를 가우시안 하나로 근사한 예시 (forward = 평균·분산 일치, reverse = 한 봉우리)

	forward KL	reverse KL
벌점 조건	데이터가 있는데 모델이 0	모델이 있는데 데이터가 0
결과	넓게 퍼짐 (mass-covering)	한 봉우리에 집중 (mode-seeking)
대표	SFT	RL · RLHF

콜드 스타트 관점: SFT는 교사의 여러 풀이 방식을 고루 품고, RL은 그중 좋은 봉우리로 좁힙니다.

SFT 없이 RL만 하면 생기는 일

◆ DeepSeek-R1-Zero는 가능성을, R1은 콜드 스타트의 필요를 보여 줍니다

R1-Zero : Base → RL

- V3-Base에 **SFT 없이** 바로 GRPO
- AIME 2024 pass@1: **15.6% → 71.0%**
- 추론 길이가 학습 중 저절로 증가
- 문제: **가독성 저하, 언어 혼합**

R1 : Base → 콜드 스타트 SFT → RL

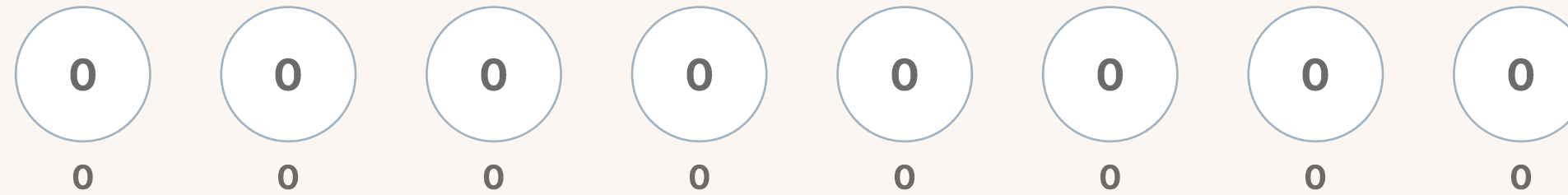
- 수천 개의 long-CoT 데이터로 먼저 SFT
- 읽기 좋은 출력 형식(요약 포함)을 고정
- 이후 추론 RL → 거절 샘플링 SFT → 전체 RL
- 형식 문제 해결 + RL 수렴이 빨라짐

RL만으로도 추론은 끌어낼 수 있지만, **형식과 가독성은 SFT가 훨씬 싸게** 해결합니다.
콜드 스타트 데이터는 '많이'보다 '원하는 형식을 정확히 보여 주는 것'이 목적입니다.

전부 0점이면 RL은 아무것도 배우지 못합니다

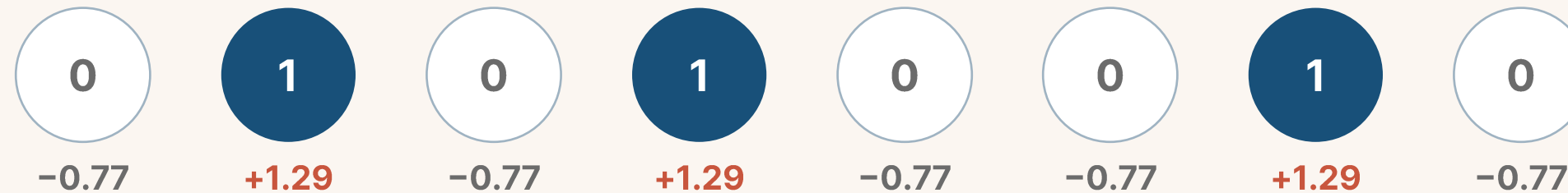
★ GRPO 어드밴티지는 그룹 안의 차이에서만 생깁니다

베이스 모델



평균 0, std 0 $\rightarrow A = 0 / (0 + \epsilon) = 0 \rightarrow \text{gradient } 0$

콜드 스타트 후



평균 0.375, std 0.484 $\rightarrow$ 성공 **+1.29**, 실패 **-0.77**

전부 1점(너무 쉬운 문제)도 같은 이유로 신호가 0입니다.

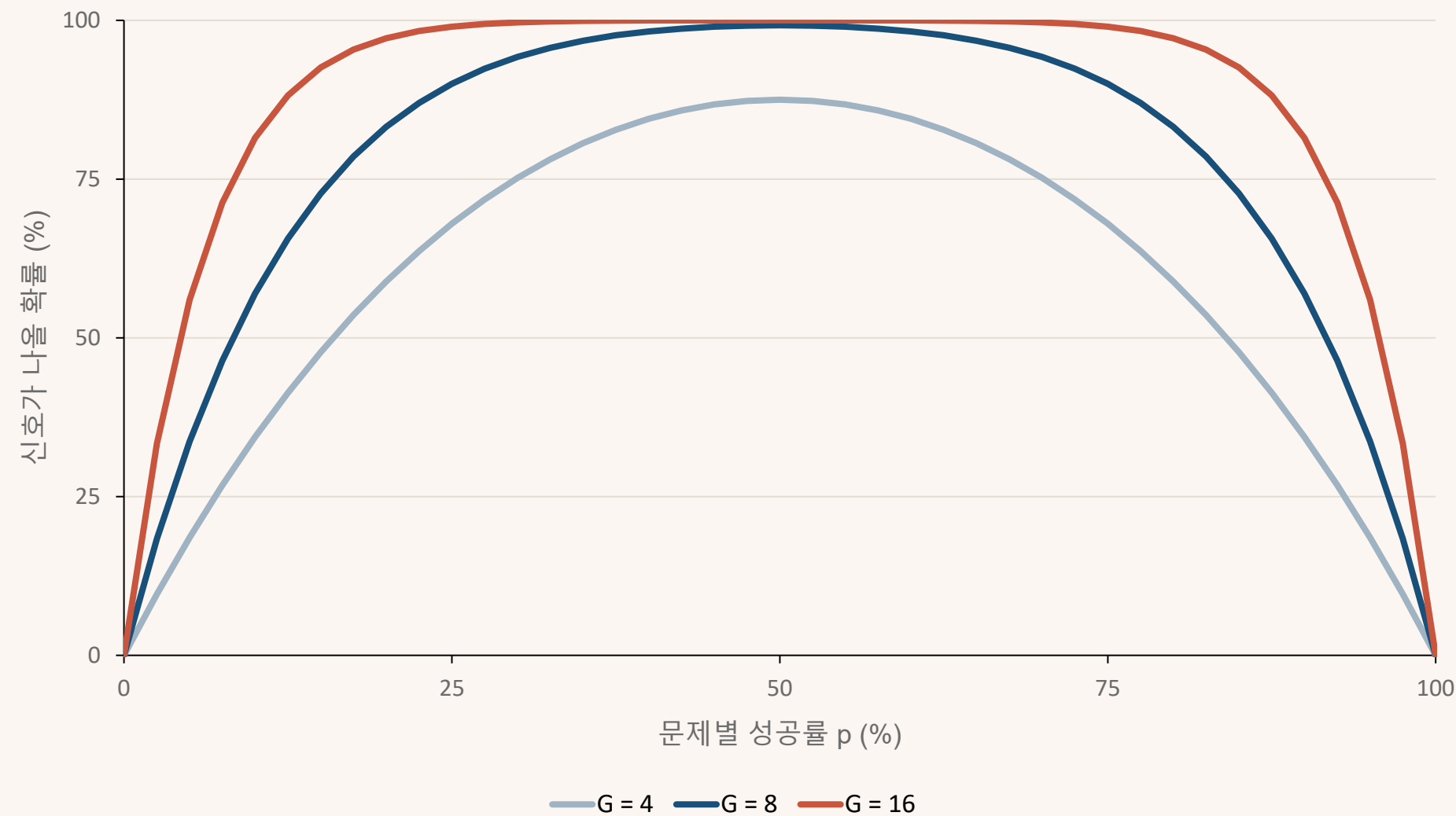
에이전트 과제는 도구 호출 형식 하나만 틀려도 0점이라, 베이스 모델에서 이 상황이 매우 흔합니다.

성공률 5%면 8개를 뽑아도 3번 중 2번은 신호 0

◆ 성공률 p 인 문제에서 그룹이 신호를 낼 확률을 계산합니다

$$P(\text{signal}) = 1 - p^G - (1 - p)^G$$

1에서 'G번 모두 성공할 확률'과 'G번 모두 실패할 확률'을 뺀 값



성공률 p	G = 4	G = 8	G = 16
1%	3.9%	7.7%	14.9%
5%	18.5%	33.7%	56.0%
10%	34.4%	57.0%	81.5%
20%	58.9%	83.2%	97.2%
50%	87.5%	99.2%	100%

콜드 스타트는 문제들을 곡선의 **가운데로 옮기는** 일입니다.
못 옮기면 G를 키워야 하고, rollout 비용이 그대로 늘어납니다.

좋은 콜드 스타트의 네 가지 조건

◆ 각 조건은 RL 쪽의 구체적인 실패를 막고, 측정할 수 있습니다

조건	RL에 왜 필요한가	확인 지표
성공률 $0 < p < 1$	그룹 안에 보상 차이가 있어야 신호가 생김	문제별 pass@1, $0 < p < 1$ 인 문제 비율
형식 준수	도구 호출 형식이 틀리면 0점, rollout 낭비	파싱 성공률, 유효 도구 호출 비율
다양성 유지	RL은 샘플끼리의 차이에서 배움	pass@k – pass@1 격차, 토큰 entropy
적정 길이 · 가독성	길이 폭주와 언어 혼합 방지, 길이 기준점	평균 응답 길이, 언어 일관성

일반 SFT의 기준은 벤치마크 점수, 콜드 스타트의 기준은 '이 체크포인트에서 RL을 돌리면 얼마나 오르나'입니다.
SFT 점수가 조금 낮아도 다양성이 살아 있는 체크포인트가 RL 후에는 더 좋을 수 있습니다 (Deep Dive에서 다룹니다).

K3: SFT → 전문가 9개 RL → 증류

◆ 2.8T MoE (활성 104B), 1M context 모델의 post-training

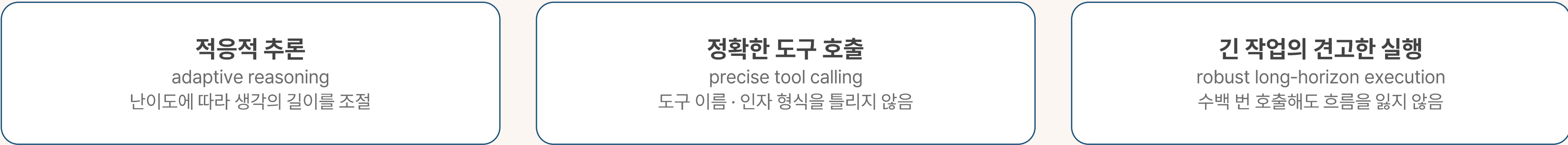


- 별도의 RLHF · DPO 단계가 없습니다. 선호 학습 대신 검증 가능한 환경에서의 RL과 증류로 구성됩니다.
- 콜드 스타트 모델은 **9개 RL이 모두 출발하는 공통 지점**이자, 뒤에서 볼 **길이 예산의 기준**입니다.
- 학습 환경은 Kimi Code, Claude Code, Codex 등 여러 에이전트 harness를 흉내 내 특정 도구 스키마에 과적합하지 않게 합니다.

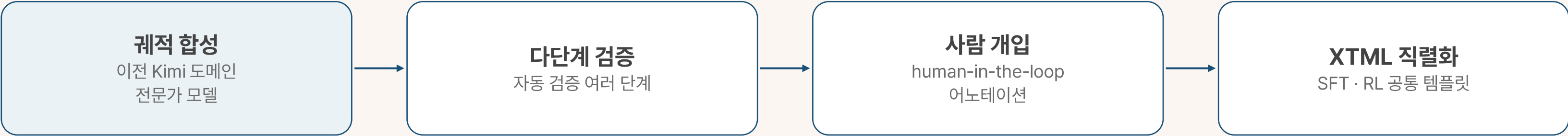
K3는 SFT를 'RL의 출발 정책'으로 정의합니다

❖ "The SFT stage establishes a high-quality cold-start policy for the subsequent RL stage."

목표 능력 3가지



데이터 파이프라인



- 교사 = 이미 RL을 거친 전문가 모델 → **증류형 콜드 스타트**. 교사의 스타일이 곧 RL 출발점이 됩니다.
- SFT 단계부터 **QAT (MXFP4 weight, MXFP8 activation)**. 배포 형식을 처음부터 맞춰 학습 · 추론 불일치를 없앱니다.

콜드 스타트 모델의 길이 습관이 RL의 길이 상한

◆ K3는 문제별 토큰 예산 b_0 를 콜드 스타트 모델로 정합니다

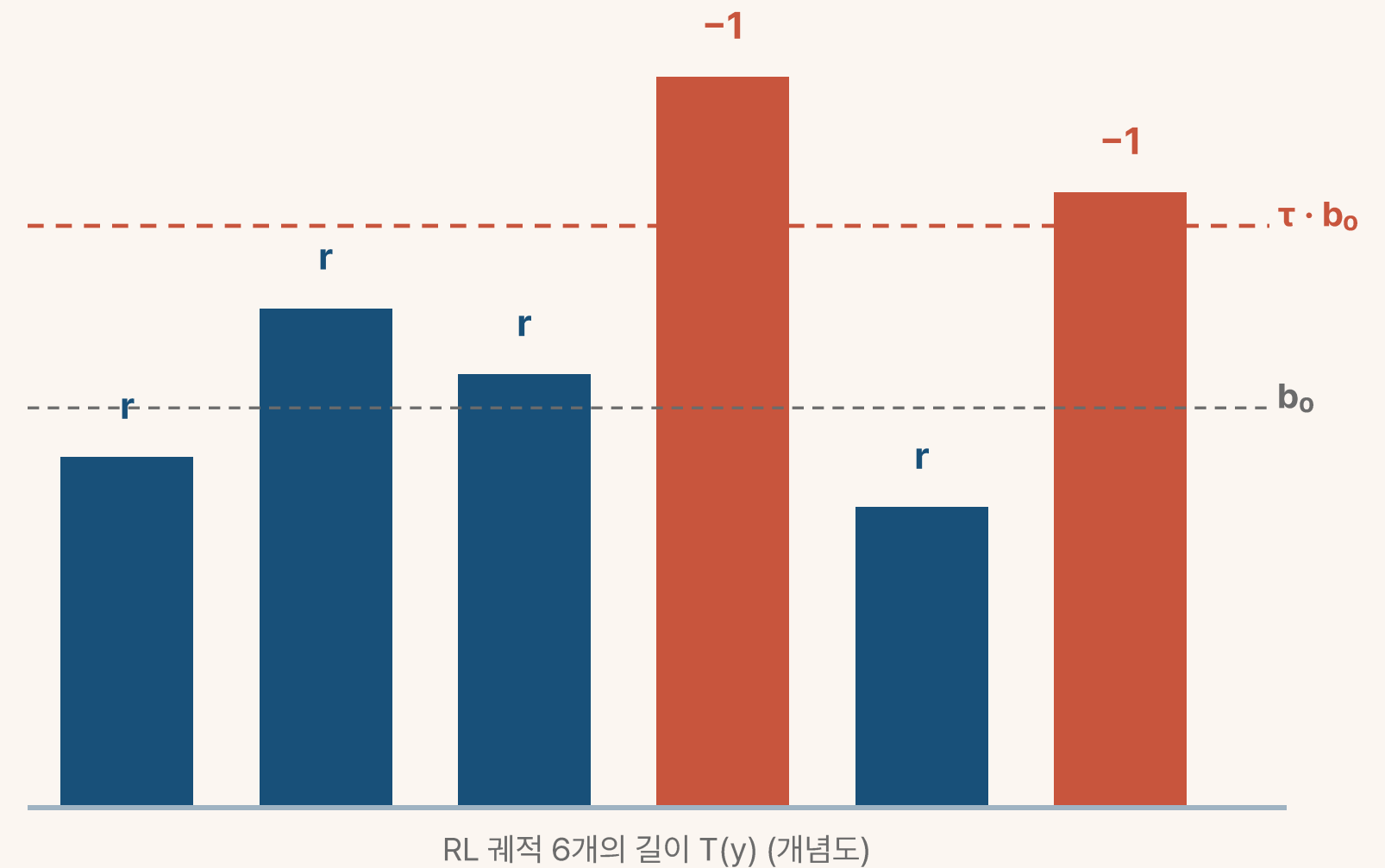
$$\tilde{r}(x, y) = -1 \quad \text{if } T(y) > \tau \cdot b_0(x)$$

$$\tilde{r}(x, y) = r(x, y) \quad \text{otherwise}$$

기호를 말로 읽기

- $b_0(x)$: 문제 x 에서 콜드 스타트 모델이 쓰는 토큰 수로 정한 예산
- τ : 예산 배율 (논문 표기, 궤적 τ 와 다른 기호)
- $T(y)$: RL 궤적 y 의 전체 토큰 수
- 예산을 넘으면 **정답이어도 -1**, 아니면 원래 보상
- low · high · max 추론 강도마다 따로 적용

시사점: 콜드 스타트 데이터의 길이 분포 = RL의 길이 제약. 난이도별 길이를 따로 점검해야 합니다.



SFT와 RL의 형식을 처음부터 맞춥니다

◆ K3의 세 가지 선택은 모두 '콜드 스타트와 RL 사이의 불일치'를 없애는 장치입니다

K3의 선택	SFT와 RL 사이에서 맞추는 것	안 맞추면 생기는 일
XHTML 채팅 템플릿	모든 궤적의 토큰 형식	RL rollout의 도구 호출 파싱 실패, 형식 불일치
SFT부터 QAT	학습 정밀도와 배포 정밀도	학습한 모델과 실제로 도는 모델의 수치 불일치
콜드 스타트 기반 예산 b_0	RL의 응답 길이 기준	길이 폭주, 추론 강도 구분 실패
전문가 모델 합성 데이터	RL 도메인과 같은 과제 분포	RL 시작 시점에 처음 보는 과제 형식

원칙: 콜드 스타트는 **RL과 같은 조건(템플릿 · 정밀도 · 과제 분포)** 에서 만들어야 합니다.
내 실험에서도 SFT 데이터 템플릿과 RL rollout 템플릿이 같은지부터 확인합니다.

궤적은 OpenAI 대화 형식으로 저장합니다

🌟 Nemotron 데이터와 NeMo-RL이 모두 쓰는 형식입니다

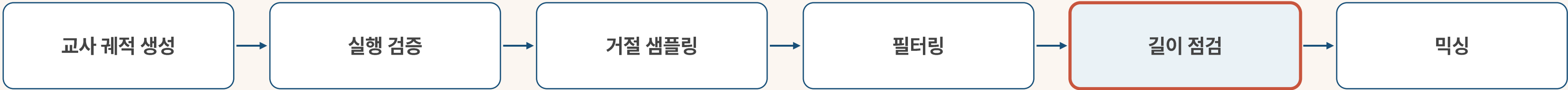
```
{
  "messages": [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "What's the weather in Paris?"},
    {"role": "assistant",
      "reasoning_content": "날씨 도구를 불러야 한다.",
      "content": "",
      "tool_calls": [{"id": "call_1", "type": "function",
        "function": {"name": "get_weather",
          "arguments": {"city": "Paris"}}}],
    {"role": "tool", "tool_call_id": "call_1",
      "content": "22°C, sunny"},
    {"role": "assistant", "content": "파리는 22°C, 맑음입니다."}
  ],
  "tools": [{"type": "function", "name": "get_weather",
    "parameters": {"type": "object",
      "properties": {"city": {"type": "string"}},
      "required": ["city"]}]
}
```

필드별 의미

- reasoning_content: 추론. loss 대상 (mask 1)
- tool_calls: 도구 이름과 인자. loss 대상
- role: tool: 환경이 준 결과. **loss 제외** (mask 0)
- tools: 사용할 수 있는 도구의 스키마. system 쪽으로 렌더링
- 마지막은 **assistant 메시지로 끝나야** 학습 대상이 생깁니다
- 실제 토큰 모양은 모델의 apply_chat_template이 정합니다

데이터 품질은 합성보다 검증에서 정해집니다

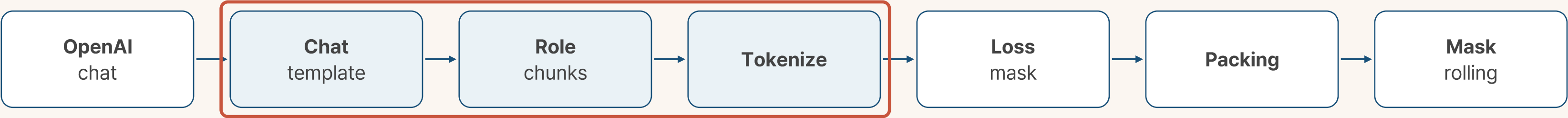
❖ 콜드 스타트 데이터는 '양보다 검증된 궤적 + RL과 같은 포맷'



단계	하는 일	놓치면 생기는 일
교사 선택	도메인 전문가 · 더 큰 모델로 궤적 생성 (K3: 이전 Kimi 전문가)	교사 스타일이 RL 출발점이 됨
실행 검증	도구 호출 파싱, 실제 실행 결과, 최종 답 정답 여부	틀린 궤적 모방 → RL이 교정부터 해야 함
거절 샘플링	문제당 여러 궤적 생성 후 통과한 것만 채택	운 좋게 맞은 궤적이 섞임
필터링	구조 오류, 반복 n-gram, 판정 모델 일관성 검사	반복 루프 · 환각 행동 학습
길이 점검	난이도별 추론 길이 분포 확인	RL 길이 기준점 왜곡 (K3 b0)
RL과 분리	콜드 스타트 프롬프트 ≠ RL 프롬프트 풀	RL 시작부터 외운 문제라 신호 없음
믹싱	일반 대화 · 수학 · 안전 데이터 일부 유지	에이전트 외 능력 망각

1. 템플릿을 입히고 역할별로 다시 자릅니다

◆ Nemotron 데이터 준비 파이프라인 7단계 중 앞부분

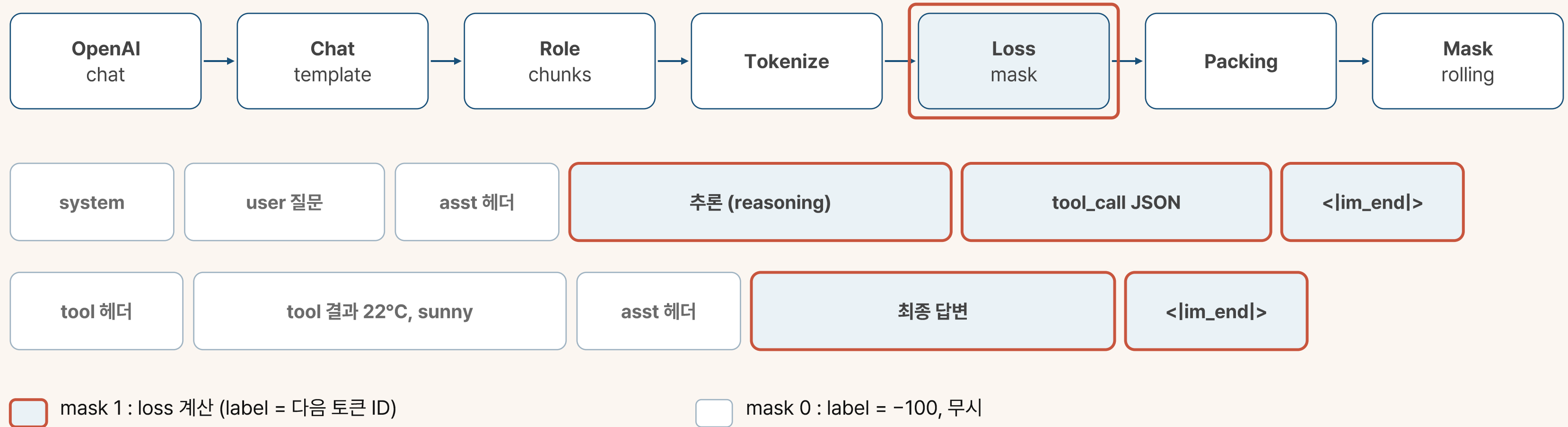


단계	하는 일	에이전트 궤적에서의 주의점
Chat template	Jinja 템플릿으로 메시지를 특수 토큰(< im_start > 등)이 붙은 텍스트로 렌더링	RL rollout과 같은 템플릿 이어야 함
Role chunks	렌더링된 텍스트를 역할(system/user/assistant/tool)이 붙은 조각으로 나눔	tool 메시지를 별도 역할로 인식해야 mask가 맞음
Tokenize	조각을 토큰 ID로 변환	조각 경계에서 BPE 분할이 전체 토큰화와 같은지 확인

Nemotron 트러블슈팅: 메시지에 <tool_call>이 있는데 system 프롬프트에 # Tools 헤더가 없으면 검증에서 실패합니다.

2. 모델이 말하는 토큰에만 loss를 겁니다

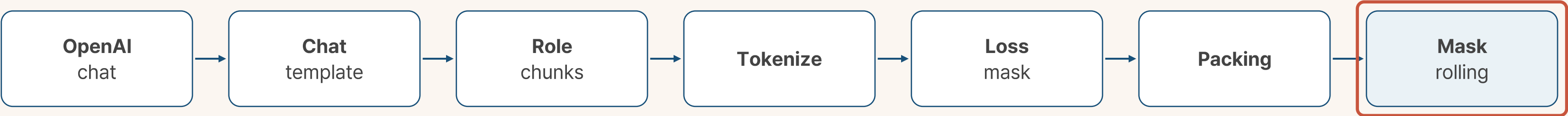
❖ 궤적 하나를 역할별 토큰 조각으로 펼쳐 봅니다



- **tool 결과는 0**: 환경이 준 것. 1로 두면 결과를 지어내는 법을 배움
- **역할 헤더는 0**: 추론 때 생성 프롬프트로 주어지는 부분
- **종료 토큰은 1**: 멈추고 도구 결과를 기다리는 법. 빠지면 RL rollout이 최대 길이까지 늘어남

3. mask를 한 칸 밀어 label 위치에 맞춥니다

❖ 위치 t 의 출력은 $t+1$ 번째 토큰을 예측하기 때문입니다



위치	0	1	2	3	4
입력	A	B	C	D	E
label (한 칸 뒤 토큰)	B	C	D	E	-
원래 mask (입력 기준)	0	0	1	1	1
밀린 mask (label 기준)	0	1	1	1	0

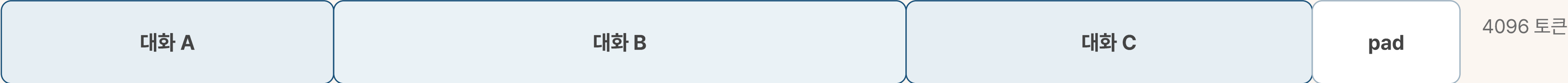
위치 1의 출력이 **C**를 예측 → C의 loss는 위치 1에
마지막 위치는 예측할 토큰이 없으니 0

코드: label 배열에 -100을 넣고 `labels[:, 1:]`로 자르면 자동으로 밀림

주의: Nemotron 문서의 예시 표는 밀린 mask를 [0,0,0,1,1]로 적고 있습니다. 이 경우 **답변 첫 토큰을 예측하는 loss가 빠집니다.**
라이브러리를 쓰더라도 label과 mask 위치를 직접 출력해 한 번은 눈으로 확인합니다.

Packing과 loss 정규화

◆ 짧은 대화를 이어 붙이고, 경계와 평균 방식을 관리합니다



seq_start_id = [0, 128, 384] → FlashAttention의 cu_seqlens로 전달, 대화 사이 attention 차단

두 가지 정규화

$$\mathcal{L}_{\text{token}} = \frac{\sum_i \sum_t m_{i,t} \ell_{i,t}}{\sum_i \sum_t m_{i,t}}$$

토큰 단위 (Nemotron 기본)
모든 답변 토큰이 같은 무게

$$\mathcal{L}_{\text{seq}} = \frac{1}{N} \sum_i \frac{\sum_t m_{i,t} \ell_{i,t}}{\sum_t m_{i,t}}$$

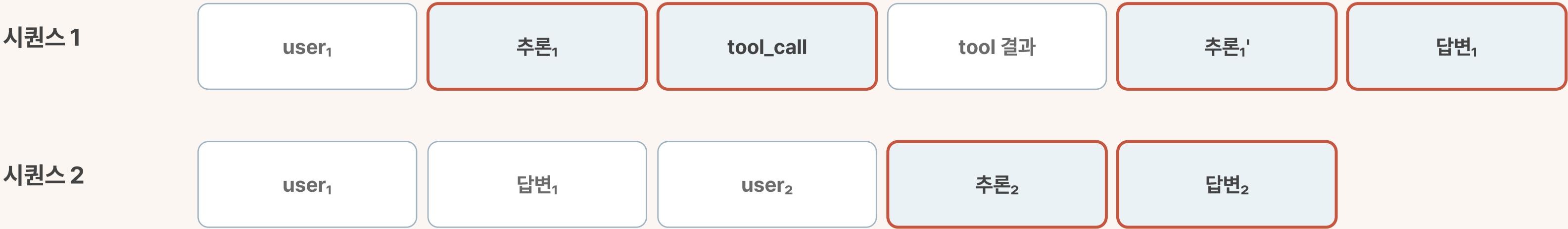
샘플 단위
짧은 샘플의 토큰이 더 무거움

실무에서 주의할 점

- 경계 간 attention 차단이 빠지면 **다른 대화를 보고 학습**
- 긴 에이전트 궤적이 섞이면 토큰 단위 평균은 **긴 궤적의 비중을 키움** → 사실상 믹싱 비율이 바뀜
- gradient 누적 시에는 **전체 배치의 mask 토큰 수**로 나눠야 토큰 단위가 유지됨
- pack 길이를 넘는 궤적은 max_doc_tokens로 관리 (끝에서 잘림)

멀티턴 추론은 user 턴마다 시퀀스를 나눕니다

✦ 추론할 때 보이지 않는 이전 턴의 추론은 학습 입력에서도 지웁니다



Nemotron 레시피로 전체 흐름 돌리기

🌟 데이터 준비 → SFT, 두 줄로 끝납니다

```
# 1) 채팅 템플릿 적용, 토큰화, loss mask, packing → Parquet
uv run nemotron lightning35 data prep sft --run YOUR-CLUSTER sample=100    # 먼저 100개로 점검

# 2) SFT (Megatron-Bridge finetune())
uv run nemotron lightning35 sft --run YOUR-CLUSTER optimizer.lr=5e-6 checkpoint.load=/path/to/base_ckpt
```

레시피 기본 하이퍼파라미터

항목	값
learning rate	5e-6 (warmup 10 iter)
global batch	128 (micro 1)
sequence length	4096 (pack size)
optimizer	AdamW
loss	assistant 토큰만, 토큰 단위 평균

먼저 확인할 것

- sample=100으로 돌리고 metadata.json의 필터·잘림 통계 확인
- 데이터 블렌드는 config/data_prep/data_blend_raw.json
- mask가 전부 0인 샘플(assistant 없음)은 빈 시퀀스로 걸러짐
- 공개 데이터만 쓰는 레시피라 결과는 모델 카드 수치와 다름

NeMo-RL에 내 에이전트 궤적 붙이기

🌟 OpenAI 형식 데이터셋을 설정 파일로 연결합니다

```
# cold_start_sft.yaml
data:
  train:
    dataset_name: openai_format
    data_path: /data/cold_start_train.jsonl
    chat_key: messages
    tool_key: tools
    use_preserving_dataset: true
  validation:
    dataset_name: openai_format
    data_path: /data/cold_start_val.jsonl
```

```
uv run examples/run_sft.py --config cold_start_sft.yaml \
  cluster.gpus_per_node=2
```

use_preserving_dataset이 필요한 이유

도구	원래 인자
search	{query: "..."}
calc	{expression: "2+2", precision: 2}

옵션이 없으면 스키마 통일 과정에서 →

도구	오염된 인자
search	{query: "...", expression: None, ...}
calc	{query: None, expression: "2+2", ...}

- 모델이 없는 인자를 넣는 도구 호출을 배움
- NeMo-RL은 messages와 tools를 apply_chat_template()에 그대로 넘김 → 렌더링은 모델 템플릿이 결정

최소 구현 (1): label 만들기

❖ 역할별로 렌더링 길이를 비교해 assistant 본문만 label로 남깁니다

```
import torch
from transformers import AutoTokenizer

tok = AutoTokenizer.from_pretrained("Qwen/Qwen3-1.7B")
IGNORE = -100 # cross_entropy가 무시하는 label 값

def render(msgs, tools, gen_prompt=False):
    return tok.apply_chat_template(msgs, tools=tools, tokenize=True, add_generation_prompt=gen_prompt)

def build_example(messages, tools):
    """user 턴 하나의 궤적 → (input_ids, labels). assistant 본문 + 종료 토큰만 label로 남김."""
    ids, labels, prev = [], [], []
    for i, msg in enumerate(messages):
        cur = render(messages[: i + 1], tools)
        # 템플릿이 앞부분을 바꾸면 (이전 추론 삭제 등) prefix 방식이 깨짐 → user 턴별로 쪼갤 것
        assert cur[: len(prev)] == prev, "prefix mismatch: split by user turn"
        new = cur[len(prev):]
        if msg["role"] == "assistant":
            n_hdr = len(render(messages[:i], tools, gen_prompt=True)) - len(prev) # "<|im_start|>assistant\n"
            labels += [IGNORE] * n_hdr + new[n_hdr:] # 헤더 0, 본문 1
        else: # system / user / tool → 전부 0
            labels += [IGNORE] * len(new)
        ids += new
        prev = cur
    return torch.tensor(ids), torch.tensor(labels)
```

최소 구현 (2): loss 계산

◆ 한 칸 밀기와 토큰 단위 평균을 코드로 확인합니다

```
import torch.nn.functional as F

def sft_loss(model, input_ids, labels, n_tok_global):
    logits = model(input_ids=input_ids).logits[:, :-1]      # 위치 t의 출력 → t+1 토큰 예측
    targets = labels[:, 1:]                                  # 한 칸 밀기 (= mask rolling)
    loss_sum = F.cross_entropy(logits.reshape(-1, logits.size(-1)), targets.reshape(-1),
                                ignore_index=IGNORE, reduction="sum")
    return loss_sum / n_tok_global                          # grad 누적 전체의 label 토큰 수로 나눔 (토큰 단위 평균)

# 학습 전 필수 점검: loss가 걸리는 토큰만 디코딩해서 눈으로 확인
ids, lab = build_example(messages, tools)
print(tok.decode(ids[lab != IGNORE]))                      # 추론 · tool_call · 최종 답 · <|im_end|> 만 보여야 정상
```

① 한 칸 밀기

`logits[:, :-1]`와 `labels[:, 1:]`가 N
emotron의 mask rolling과 같은 일

② 토큰 단위 평균

`n_tok_global` = grad 누적 전체의 label 토큰
수. 마이크로배치 평균은 짧은 배치를 과대 반영

③ 종료 토큰

`<|im_end|>`가 label에 있어야 멈추는 법을 배움.
빠지면 rollout이 최대 길이까지

SFT는 형식을, RL은 일반화를 가져옵니다

✦ Chu et al. (2025), SFT Memorizes, RL Generalizes

관찰

- 규칙 · 시각 조건을 바꾼 분포 밖 과제로 비교
- 결과 보상 RL: **바뀐 규칙에도 일반화**
- SFT: 학습 데이터를 **외우는 경향**이 강함

같은 논문의 단서

- RL이 효과적으로 돌려면
- **SFT로 출력 형식을 먼저 안정시켜야 했음**
- SFT는 RL의 경쟁자가 아니라 전제 조건

콜드 스타트에 주는 의미: **형식은 SFT로 싸게 고정**하고, 과제 해결 능력의 일반화는 RL에 맡깁니다.
그래서 콜드 스타트는 '많이, 오래' 학습할 이유가 약합니다. 형식이 잡히면 넘어가는 편이 낫습니다.

RL은 가끔 맞던 것을 자주 맞히게 만듭니다

★ Yue et al. (2025): 콜드 스타트가 pass@k 천장을 정합니다

$$\text{pass}@k = \mathbb{E}_x \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right]$$

기호를 말로 읽기 (Chen et al. 2021 비편향 추정)

- 문제마다 n개를 뽑아 c개가 맞았을 때
- 'k개를 골랐는데 전부 틀릴 확률'을 1에서 뺀 값
- k개 중 **하나라도** 맞으면 성공으로 보는 지표

	k가 작을 때	k가 클 때
RLVR 모델	앞섬	base가 따라잡거나 역전
의미	자주 맞힘	새 풀이는 적음

- RL 모델의 추론 경로는 대부분 **base 샘플 분포 안에 이미 있음**
- 강한 교사로부터의 **종류는 새 추론 패턴**을 들여올 수 있음

콜드 스타트 (특히 증류형)가 pass@k 천장을 올리고, RL은 그 천장 아래에서 pass@1을 끌어올립니다.
체크포인트를 고를 때 pass@1만 보지 말고 pass@8, pass@32를 같이 봅니다.

SFT를 너무 오래 하면 탐색할 여지가 줄어듭니다

★ Cui et al. (2025): RL 성능은 정책 **entropy**를 소모하며 얻어집니다

$$R \approx -a e^H + b$$

R: 성능, H: 정책 entropy, a · b: 모델 · 데이터마다 맞춘 상수
entropy가 0으로 떨어지면 성능 상한은 $-a + b$ 로 정해짐

메커니즘

- RL은 샘플끼리의 차이에서 배움 → 다양성이 곧 연료
- entropy가 초반에 빠르게 무너지면 성능이 **일찍 정체**
- 과한 SFT는 RL을 낮은 **entropy**에서 시작하게 만듦

과적합 신호 3가지

- 학습 loss는 내려가는데 **검증 loss가 오르기 시작**
- 같은 프롬프트로 8번 뽑은 답이 **거의 동일**
- pass@1은 오르는데 **pass@k가 정체하거나 하락**

대응: 1~2 에폭으로 제한하고, 에폭별 체크포인트의 entropy와 pass@k를 기록해 두었다가 RL probe로 고릅니다.

1.7B가 따라 할 수 있는 궤적인가

◆ Li et al. (2025): 작은 모델은 강한 교사의 긴 추론을 잘 배우지 못합니다

	관찰	콜드 스타트에서의 대응
긴 CoT	작은 모델은 긴 추론을 모방하면 오히려 성능 저하	짧은 궤적과 긴 궤적을 섞기 (Mix Distillation)
큰 교사	교사와 학생의 크기 차이가 클수록 배우기 어려움	학생 크기에 가까운 교사도 섞기
학생 분포	교사 궤적은 학생이 낼 법한 분포와 멀 수 있음	학생이 직접 뽑고 통과한 궤적으로 재SFT

좋은 궤적 ≠ 학생이 배울 수 있는 궤적. 교사 품질만 올려서는 콜드 스타트가 좋아지지 않을 수 있습니다.
작은 모델 실험(예: 1.7B)에서는 교사 궤적의 길이 분포를 학생 기준으로 다시 맞추는 것부터 확인합니다.
K3는 교사가 같은 계열의 전문가 모델이라는 점에서 이 격차가 상대적으로 작은 설정입니다.

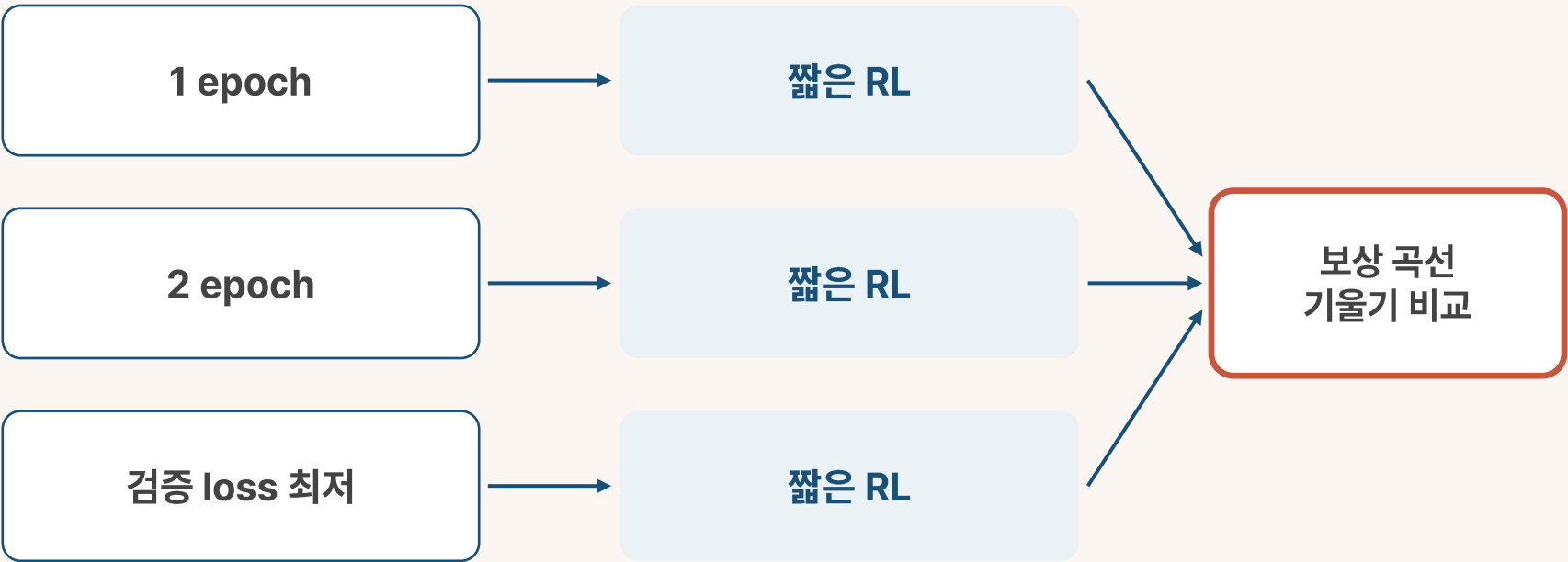
체크포인트는 loss가 아니라 RL probe로 고릅니다

❖ 콜드 스타트 모델은 RL의 출발점이자 KL의 닳입니다

$$\mathcal{J}_{\text{RLHF}} = \mathbb{E}_{y \sim \pi_{\theta}}[R(y)] - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$$

π_{ref} = 콜드 스타트 모델 → RL이 너무 멀리 가지 못하게 붙잡는 닳. 에이전트 데이터만 넣으면 일반 능력이 약한 상태가 닳습니다.

RL probe로 고르기



같은 RL 설정 · 같은 step 수로 돌리고, 초반 보상 상승 기울기와 entropy 추이를 비교합니다.

시작점 하이퍼파라미터 (경험칙)

항목	시작값 · 메모
learning rate	5e-6 전체 파인튜닝 (Nemotron 기본), 작은 모델은 1e-5까 지
epoch	1~2 (과하면 entropy 손실)
global batch	128 시퀀스
seq length	궤적 길이 분포의 95% 지점
평가	검증 loss + 형식 준수율 + pass@k + RL probe

오늘의 정리

★ 콜드 스타트 SFT = RL이 배울 수 있는 상태로 모델을 옮기는 일

1

같은 식의 두 극단

SFT gradient = 궤적을 모범 데이터로, 어드밴티지를 1로 둔 policy gradient. SFT는 forward KL로 넓게 덮고, RL은 좁힌다.

2

신호는 차이에서

GRPO는 그룹 안에 성공과 실패가 섞여야 배운다. 성공률 5%, $G=8$ 이면 3번 중 2번은 신호 0.

3

K3의 정의

SFT = RL을 위한 고품질 cold-start policy. 전문가 모델 합성 + 검증, XTM, QAT, 그리고 RL 길이 예산의 기준.

4

데이터와 마스크

tool 결과 0, 종료 토큰 1, mask는 label 기준으로 한 칸. 템플릿은 RL rollout과 같게.

5

고르는 법

형식이 잡히면 멈춘다. pass@k와 entropy를 보고, 최종 선택은 RL probe로.

학습 전 체크리스트와 흔한 실수

대부분의 실수는 마스크와 템플릿에서 나옵니다

체크리스트

- SFT와 RL rollout의 템플릿 · 도구 스키마가 같은가
- 마스크 덤프로 tool 결과 · 헤더 0, 종료 토큰 1을 확인했는가
- 모든 궤적이 실행 검증을 통과했는가
- 콜드 스타트와 RL 프롬프트가 겹치지 않는가
- 난이도별 추론 길이 분포를 봤는가
- packing 경계의 attention이 막혀 있는가
- pass@1과 pass@k, entropy를 에폭별로 기록했는가
- $0 < p < 1$ 인 RL 문제 비율이 늘었는가

흔한 실수	증상	해결
tool 결과에 loss	결과를 지어냄	tool 역할 mask 0
종료 토큰 mask 0	멈추지 않음	종료 토큰 포함
이전 턴 추론 학습	추론 시 입력과 불일치	user 턴 단위 분할
스키마 자동 정규화	None 인자 도구 호출	preserving dataset
마이크로배치 평균	짧은 샘플 과대 반영	전체 토큰 수로 나눔
과한 에폭	entropy 붕괴, 보상 정체	1~2 에폭 + RL probe

함께 생각해 볼 질문

★ 정답이 정해지지 않은, 실험으로 확인할 문제들입니다

Q1 데이터는 얼마나 필요한가?

R1은 수천 개로 시작했고 K3는 대규모 합성 데이터를 썼습니다. 형식만 잡는 데 필요한 양과 능력까지 심는 데 필요한 양은 다를까요?

Q2 교사 궤적 vs 학생 자신의 궤적

강한 교사의 증류는 천장을 올리지만 배우기 어렵고, 학생의 거절 샘플링은 배우기 쉽지만 천장이 낮습니다. 어떻게 섞을까요?

Q3 에이전트 RL에도 SFT가 꼭 필요한가?

R1-Zero처럼 base에서 바로 시작하는 것이 에이전트 과제에서도 가능할까요? 형식 보상만으로 충분할까요?

Q4 어디까지 mask 0인가?

tool 결과는 0이 맞지만, 실패한 도구 호출과 그 뒤의 복구 과정은 학습해야 할까요, 걸러야 할까요?

참고 문헌

- Kimi Team. Kimi K3: Open Frontier Intelligence. arXiv:2607.24653, 2026.
- DeepSeek-AI. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948, 2025.
- NVIDIA. Nemotron 3.5 Lightning, Stage 1: Supervised Fine-Tuning (SFT). docs.nvidia.com/nemotron, 2026.
- NVIDIA. Supervised Fine-Tuning in NeMo RL. github.com/NVIDIA-NeMo/RL, docs/guides/sft.md.
- Chu et al. SFT Memorizes, RL Generalizes: A Comparative Study of Foundation Model Post-training. arXiv:2501.17161, 2025.
- Yue et al. Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model? arXiv:2504.13837, 2025.
- Cui et al. The Entropy Mechanism of Reinforcement Learning for Reasoning Language Models. arXiv:2505.22617, 2025.
- Li et al. Small Models Struggle to Learn from Strong Reasoners. arXiv:2502.12143, 2025.
- Liu et al. Understanding R1-Zero-Like Training: A Critical Perspective (Dr. GRPO). arXiv:2503.20783, 2025.
- Chen et al. Evaluating Large Language Models Trained on Code (pass@k estimator). arXiv:2107.03374, 2021.
- N. Lambert. Post-Training Course, Lecture 1 & RLHF Book. rlhfbook.com, 2026.

상태를 정하면 다음을 계산할 수 있어요

날씨를 예로 작은 규칙을 만들어 봅니다

- 먼저 상태를 정합니다

상태는 지금 상황을 간단히 적은 이름입니다. 오늘 날씨를 '맑음' 또는 '비'로 표시하겠습니다.
내일은 오늘 상태와 정해 둔 확률에 따라 바뀝니다.

- 시간이 바뀔 때는 이렇게 움직입니다

오늘 상태	내일 맑음	내일 비
맑음	0.8	0.2
비	0.4	0.6

같은 오늘 상태라면, 이 모델은 그보다 오래된 날씨 기록을 다시 보지 않습니다.

마르코프 성질은 '지금만 보기'

◆ 현재 상태를 알고 나면 더 오래된 기록은 다시 쓰지 않습니다

- 말로 먼저 이해하기

$$\begin{aligned} & 1. P(X_{t+1} = j \mid X_t = i, X_{t-1}, \dots, X_0) \\ &= P(X_{t+1} = j \mid X_t = i) \end{aligned}$$

1. 현재 상태를 알고 난 뒤에는 과거 정보가 다음 상태 확률을 더 바꾸지 않습니다.

오늘이 맑다는 사실을 알았다면, 내일 맑거나 비일 확률은 오늘 상태만으로 정합니다.
어제와 그제 날씨는 이 단순한 모델에서 추가로 보지 않습니다.

전이행렬은 한 칸 이동 확률표

◆ 행은 지금, 열은 다음 상태를 뜻합니다

• 행과 열의 뜻

상태 순서를 [맑음, 비]로 정합니다.
맑음 행에서 0.8은 내일 맑음, 0.2는 내일 비일 확률입니다.

• 현재 상태별 다음 상태 확률

현재 \ 다음	맑음	비
맑음	0.8	0.2
비	0.4	0.6

한 행은 현재 상태 하나의 모든 다음 가능성을 담으므로, 행의 합은 1입니다.

현재 분포에 P를 곱하면 다음 분포

◆ 처음에는 확실히 맑다고 해 봅시다

- 맑음일 확률을 계산

다음 맑음 = 지금 맑음 × 0.8 + 지금 비 × 0.4
= 1 × 0.8 + 0 × 0.4 = 0.8

- 한 걸음 뒤의 분포

시점	맑음 확률	비 확률
처음 π_0	1.00	0.00
1일 뒤 π_1	0.80	0.20

그래서 $\pi_1 = \pi_0 P = (0.8, 0.2)$ 입니다.

두 번째 날도 같은 방식으로 계산

◆ 첫날이 맑음 0.8, 비 0.2라면?

- 각 현재 상태가 만드는 몫을 더합니다

다음 맑음에는 맑음에서 오는 0.64와 비에서 오는 0.08이 더해집니다.
다음 비에도 두 경우의 몫을 더합니다.

- 둘째 날 확률을 한 줄씩 만들기

첫날 상태	둘째 날 맑음에 기여	둘째 날 비에 기여
맑음 0.8	$0.8 \times 0.8 = 0.64$	$0.8 \times 0.2 = 0.16$
비 0.2	$0.2 \times 0.4 = 0.08$	$0.2 \times 0.6 = 0.12$
합계	$0.64 + 0.08 = 0.72$	$0.16 + 0.12 = 0.28$

$\pi_2 = (0.72, 0.28)$. 두 확률을 더하면 다시 1입니다.

마르코프 체인 · 오래 지난 뒤

정상분포는 매일 그대로인 비율

◆ 한 걸음 지나도 바뀌지 않는 확률을 찾습니다

- 맑음 비율을 s , 비 비율을 r 로 놓기

1. 1) $s - 0.8s = 0.4r$, 그래서 $0.2s = 0.4r$

2) 양쪽을 0.2로 나누면 $s = 2r$

3) $s + r = 1$ 에 넣으면 $3r = 1$

1. 4) $r = 1/3, s = 2/3$

정상분포는 $(2/3, 1/3)$ 입니다.

정상분포 $\pi = (s, r)$ 는 $\pi P = \pi$ 를 만족합니다.

맑음 비율은 다음 날에도 s 여야 합니다.

$$s = 0.8s + 0.4r$$

또 전체 확률이므로 $s + r = 1$

마르코프 체인 · 오래 지난 뒤

시작이 달라도 한 비율에 가까워져요

◆ 정상분포를 실제 반복 계산과 비교합니다

- 시작은 맑음 확률 1

앞에서 찾은 정상분포는 맑음 2/3, 비 1/3입니다.
매일 P를 곱하면 이 값에 가까워지는지 확인합니다.
이 예는 두 날씨가 서로 오가고, 같은 날씨에 머물 수도 있습니다.

- 하루씩 계산한 분포

날짜	맑음	비
0일	1.0000	0.0000
1일	0.8000	0.2000
2일	0.7200	0.2800
3일	0.6880	0.3120
오래 뒤	0.6667	0.3333

다시 돌아오는 평균 시간

장기 비율을 알면 평균 재방문 시간을 알 수 있습니다

공식이 말하는 것

유한하고 기약적인 체인에서 상태 i 의 평균 첫 재방문 시간은 $1/\pi_i$ 입니다.
처음 i 에 있는 순간부터 세고, 한 번 이상 움직인 뒤 다시 i 에 올 때까지 셉니다.

날씨 예제에 적용

상태	장기 비율 π_i	평균 재방문 시간
맑음	2/3	$1 \div (2/3) = 1.5\text{일}$
비	1/3	$1 \div (1/3) = 3\text{일}$

1.5일은 매번 반 걸음 간다는 뜻이 아니라, 여러 번의 재방문 시간을 평균낸 값입니다.

도달 시간과 흡수 시간은 다릅니다

◆ 먼저 어떤 상태에 도착하는지 구별합니다

- 비슷한 말을 따로 읽기
문제를 풀기 전에 목표 상태와 멈추는 상태를 표시합니다.
목표에 도착하는 것과 게임이 끝나는 것이 늘 같지는 않습니다.

- 세 가지 상태 용어

말	쉬운 뜻	예
도달 시간	목표 상태를 처음 만날 때까지	처음 4달러에 도달
흡수 상태	들어가면 더는 나오지 않는 상태	0달러 또는 목표 금액
흡수 시간	흡수 상태에 처음 들어갈 때까지	파산 또는 승리까지

목표가 여러 개라면 어느 목표에 먼저 닿는지도 확률로 계산할 수 있습니다.

첫 한 걸음부터 식을 세웁니다

◆ 모르는 값을 다음 상태들의 값으로 바꿉니다

- 확률과 평균 시간은 식이 조금 다릅니다

1. 기대 도달 시간 h_i :

$$h_i = 1 + \sum_j P_{ij} h_j$$

목표에서는 $h_a = 0$ 입니다. 앞의 1은 첫 걸음입니다.

1. 작은 체인은 연립방정식으로, 규칙 있는 체인은 대칭으로 풀니다.

목표에 도달할 확률 q_i :

$$q_i = \sum_j P_{ij} q_j$$

목표 상태 A에서는 $q_a = 1$ 입니다.

도달하면 안 되는 끝 상태 B에서는 $q_\beta = 0$ 입니다.